

PROGRAMAÇÃO FRONT-END

PERFORMANCE EM SITES FRONT-END



SEMANA 15
3º BIMESTRE

PERFORMANCE EM SITES DE VENDAS

Um site de vendas precisa ser rápido, leve e fácil de usar.

Quando uma página demora para carregar, o cliente pode desistir antes mesmo de visualizar os produtos. Por isso, os desenvolvedores utilizam técnicas e ferramentas para melhorar a performance do site.

Neste conteúdo, vamos estudar:

- Lazy Loading;
- minificação de CSS;
- Google Lighthouse.

O QUE É PERFORMANCE?

Performance é a capacidade de um site carregar e funcionar com rapidez.

Um site com boa performance:

- abre rapidamente;
- carrega as imagens sem travar;
- responde aos cliques do usuário;
- funciona bem em computadores e celulares;
- utiliza os arquivos de forma eficiente.

Em um site de vendas, uma boa performance ajuda o cliente a visualizar os produtos e concluir sua compra com mais facilidade.



POR QUE UM SITE PODE FICAR LENTO?

Um site pode ficar lento quando possui:

- muitas imagens pesadas;
- arquivos CSS muito grandes;
- arquivos desnecessários;
- muitos recursos carregando ao mesmo tempo;
- códigos pouco otimizados.

Imagine uma loja virtual com 100 produtos.

Se todas as 100 imagens forem carregadas ao mesmo tempo, o navegador terá muito trabalho. Isso pode deixar a página lenta.



O QUE É LAZY LOADING?

Lazy Loading é uma técnica que carrega imagens e outros recursos somente quando eles estão próximos da área visível do usuário.

O nome pode ser traduzido como:

Carregamento preguiçoso.

Isso significa que o navegador não carrega tudo de uma única vez.

As imagens que estão mais abaixo na página esperam até o usuário chegar perto delas.

Loading...



EXEMPLO DE LAZY LOADING

Imagine um site com 30 produtos.

Quando o cliente entra na página, ele consegue visualizar apenas os primeiros produtos.

Com Lazy Loading:

- as primeiras imagens são carregadas;
- as imagens mais abaixo ficam esperando;
- quando o cliente rola a página, novas imagens são carregadas.

Assim, o navegador realiza menos trabalho no início.



LAZY LOADING NO SITE DE VENDAS

Sem Lazy Loading:

```

```

Com Lazy Loading:

```
<div class="produto">  
    
  <h2>Produto 1</h2>  
  <p>R$ 100,00</p>  
  <button onclick="comprar('Produto 1')">Comprar</button>  
</div>
```

O comando responsável pelo Lazy Loading é:

```
loading="lazy"
```

ANALOGIA DO LAZY LOADING

O Lazy Loading funciona como um funcionário de uma loja.

Ele não coloca todos os produtos do estoque na frente do cliente ao mesmo tempo.

Os produtos são apresentados aos poucos, conforme o cliente avança pela loja.

No site, as imagens também são carregadas aos poucos, conforme o usuário rola a página.

BENEFÍCIO DO LAZY LOADING

O principal benefício do Lazy Loading é evitar que todas as imagens sejam carregadas ao mesmo tempo.

Isso pode:

- melhorar o carregamento inicial;
- reduzir o uso de internet;
- diminuir o trabalho do navegador;
- deixar a página mais rápida;
- melhorar a experiência do usuário.



CUIDADO COM O LAZY LOADING

O Lazy Loading é mais indicado para imagens que estão abaixo da área inicialmente visível.

Exemplos:

- produtos no final da página;
- imagens de outras seções;
- fotos que aparecem após rolar o site.

Imagens importantes que aparecem logo no início podem carregar normalmente.

Exemplos:

- logotipo;
- banner principal;
- imagem principal do primeiro produto.



O QUE É MINIFICAÇÃO DE CSS?

Minificação é o processo de reduzir o tamanho de um arquivo CSS.

Durante esse processo, são removidos elementos que não são necessários para o navegador, como:

- espaços;
- quebras de linha;
- comentários;
- formatações usadas apenas para facilitar a leitura.

O funcionamento visual do site continua o mesmo.



CSS ORGANIZADO

Durante o desenvolvimento, o CSS pode ser escrito assim:

```
.produto {  
  width: 250px;  
  margin: 20px;  
  padding: 15px;  
  text-align: center;  
}
```

Esse formato é organizado e fácil de ler.

Ele ajuda o desenvolvedor a localizar e modificar os comandos.



CSS MINIFICADO

O mesmo código pode ficar assim:

```
.produto{width:250px;margin:20px;padding:15px;text-align:center}
```

Os dois códigos produzem o mesmo resultado visual.

A diferença é que o segundo ocupa menos espaço porque foi minificado.

```
h2 {font-size: 15px; margin-top: 12px;}
```

O QUE A MINIFICAÇÃO REMOVE?

Antes da minificação:

```
/* Estilo do botão de compra */
```

```
.botao-comprar {  
    background-color: green;  
    color: white;  
    padding: 10px;  
}
```

Depois da minificação:

```
.botao-comprar{background-color:green;color:white;padding:10px}
```

O comentário, os espaços e as quebras de linha foram removidos.

O botão continua funcionando e mantendo a mesma aparência.



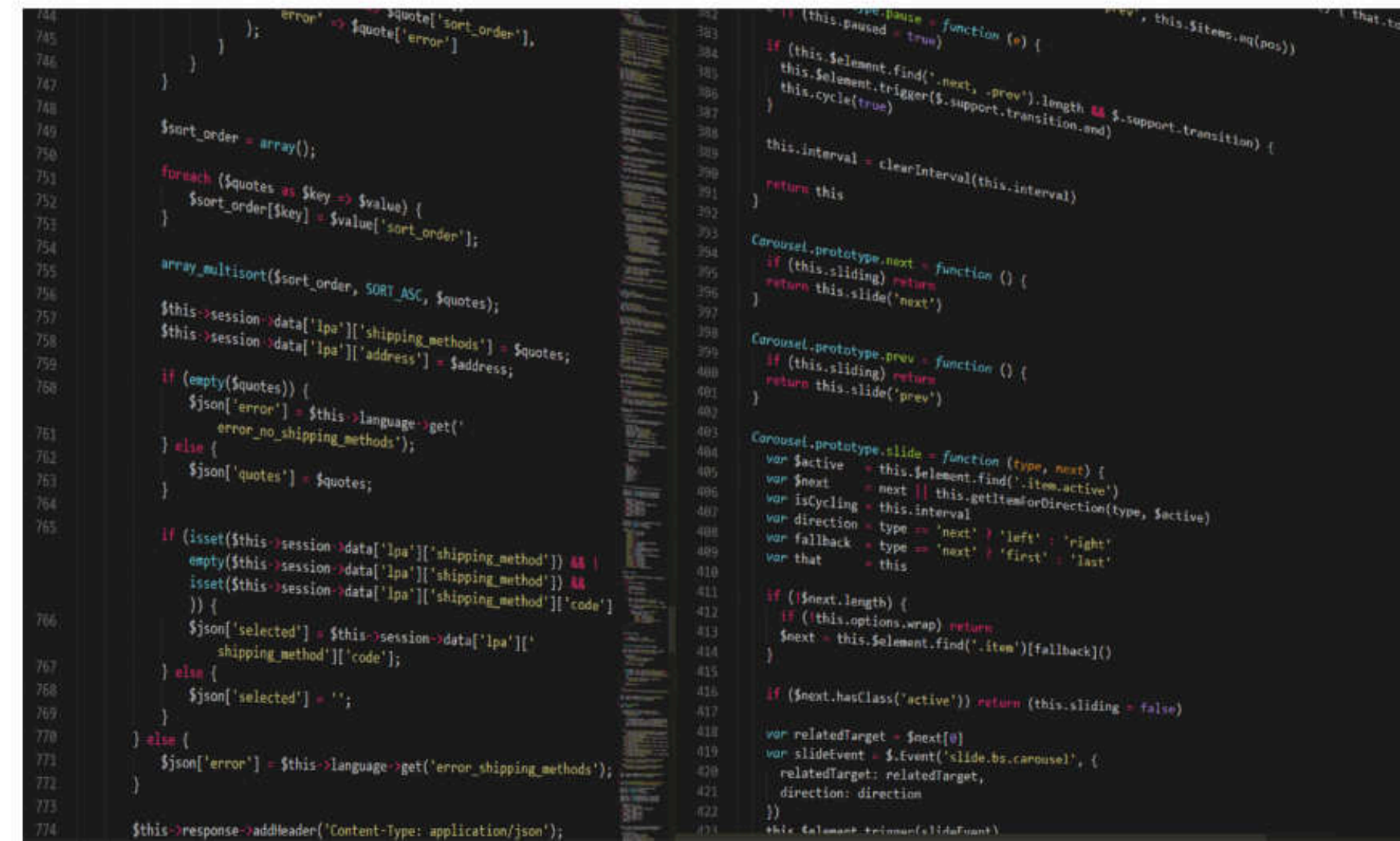
BENEFÍCIO DA MINIFICAÇÃO DE CSS

O principal benefício da minificação é reduzir o tamanho do arquivo CSS.

Um arquivo menor pode ser transferido mais rapidamente para o navegador.

Isso ajuda a:

- diminuir o tempo de carregamento;
- reduzir o tamanho dos arquivos;
- melhorar a performance;
- deixar o site mais leve.



The image shows a screenshot of a code editor with two columns of code. The left column contains minified CSS code, including rules for sorting and session data. The right column contains minified JavaScript code, including a jQuery plugin for a carousel. The code is color-coded and includes line numbers on the left margin.

```
744     error' => $quote['sort_order'],  
745     );  
746 }  
747 }  
748  
749 $sort_order = array();  
750  
751 foreach ($quotes as $key => $value) {  
752     $sort_order[$key] = $value['sort_order'];  
753 }  
754  
755 array_multisort($sort_order, SORT_ASC, $quotes);  
756  
757 $this->session->data['lpa']['shipping_methods'] = $quotes;  
758 $this->session->data['lpa']['address'] = $address;  
759  
760 if (empty($quotes)) {  
761     $json['error'] = $this->language->get('error_no_shipping_methods');  
762 } else {  
763     $json['quotes'] = $quotes;  
764 }  
765  
766 if (isset($this->session->data['lpa']['shipping_method']) && !  
767     empty($this->session->data['lpa']['shipping_method']) && !  
768     isset($this->session->data['lpa']['shipping_method']['code'])  
769 ) {  
770     $json['selected'] = $this->session->data['lpa']['shipping_method']['code'];  
771 } else {  
772     $json['selected'] = '';  
773 }  
774 } else {  
775     $json['error'] = $this->language->get('error_shipping_methods');  
776 }  
777  
778 $this->response->addHeader('Content-Type: application/json');  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

ARQUIVOS CSS MINIFICADOS

É comum encontrar arquivos com estes nomes:

`style.css`

`style.min.css`

O arquivo `style.css` é a versão organizada, utilizada durante o desenvolvimento.

O arquivo `style.min.css` é a versão reduzida, utilizada principalmente quando o site é publicado.



O QUE É GOOGLE LIGHTHOUSE?

Google Lighthouse é uma ferramenta utilizada para analisar a qualidade de uma página.

Ela examina o site e gera um relatório com pontuações, problemas encontrados e sugestões de melhorias.

O Lighthouse pode analisar áreas como:

- performance;
- acessibilidade;
- boas práticas;
- SEO.



Google
Lighthouse

LIGHTHOUSE NO SITE DE VENDAS

Em um site de vendas, o Lighthouse pode identificar:

- imagens muito pesadas;
- recursos que demoram para carregar;
- problemas de acessibilidade;
- textos difíceis de visualizar;
- elementos que prejudicam a velocidade;
- oportunidades de otimização.



Depois da análise, a ferramenta mostra recomendações para melhorar o site.

ANALOGIA DO LIGHTHOUSE

O Lighthouse funciona como uma avaliação do site.

Ele analisa diferentes partes da página e apresenta uma pontuação.

Além da nota, ele mostra o que pode ser melhorado.

É parecido com uma correção de atividade que informa:

- o que está correto;
- o que possui problema;
- o que precisa ser melhorado.



RELATÓRIO DO LIGHTHOUSE

Um relatório pode apresentar resultados como:

Performance: 70

Acessibilidade: 90

Boas práticas: 85

SEO: 92



Uma pontuação baixa em performance pode indicar que a página está demorando para carregar.

O relatório também pode sugerir melhorias.



Performance



Accessibility



Best Practices



SEO



Progressive
Web App

0-49 50-89 90-100



Performance

Metrics



● First Contentful Paint

1.0 s

● First Meaningful Paint

1.5 s

● Speed Index

1.6 s

● First CPU Idle

2.4 s

● Time to Interactive

3.0 s

■ Max Potential First Input Delay

200 ms

Values are estimated and may vary.

POSSÍVEIS RECOMENDAÇÕES

O Lighthouse pode recomendar:

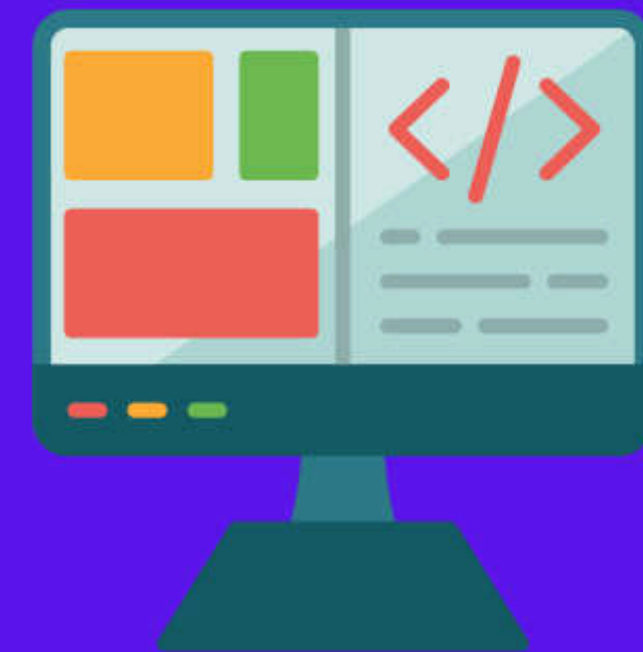
- reduzir o tamanho das imagens;
- utilizar Lazy Loading;
- diminuir arquivos CSS;
- remover recursos desnecessários;
- melhorar a acessibilidade;
- otimizar o carregamento da página.



O objetivo não é apenas conseguir uma nota alta.

O mais importante é entender os problemas apresentados e melhorar o site.

SASS E SCSS



SEMANA 16
3º BIMESTRE

SASS E SCSS — ORGANIZANDO MELHOR O CSS

Nosso site de vendas já possui várias páginas:

- Início;
- Produtos;
- Contato.

Cada página possui seus próprios estilos.

Quanto maior o site fica, maior também fica a quantidade de CSS que precisamos organizar e manter.

É nesse momento que ferramentas como o **SASS** podem ajudar.

1. O QUE É SASS?

SASS é uma ferramenta que adiciona recursos extras ao CSS.

Podemos pensar assim:

CSS = estilização normal do site

SASS = CSS com ferramentas extras para organização e reutilização

O SASS ajuda principalmente quando o projeto começa a ficar maior e possui muitos estilos repetidos.



UM PROBLEMA NO NOSSO SITE

No nosso site usamos várias vezes cores parecidas:

```
background-color: #111111;  
background-color: #000000;  
background-color: #1a1a1a;  
color: white;
```

Também repetimos vários efeitos:

```
transition: 0.3s;
```

E estilos semelhantes aparecem em páginas diferentes.

Se decidirmos trocar uma cor usada em vários lugares, talvez seja necessário procurar e modificar diversos trechos do CSS.

Em projetos grandes, isso pode dificultar a manutenção.

2. VARIÁVEIS NO SASS

Com SASS podemos criar uma variável para guardar um valor que será reutilizado.

Exemplo:

```
$cor-fundo: #111111;  
$cor-card: #1a1a1a;  
$cor-texto: white;
```

Depois podemos utilizar essas variáveis:

```
body {  
  background-color: $cor-fundo;  
  color: $cor-texto;  
}  
  
.produto {  
  background-color: $cor-card;  
}
```

Agora imagine que queremos mudar a cor principal do site.

Em vez de procurar todos os lugares onde a cor aparece, podemos alterar apenas:

```
$cor-fundo: #202020;
```

Os locais que utilizam essa variável passam a receber o novo valor.

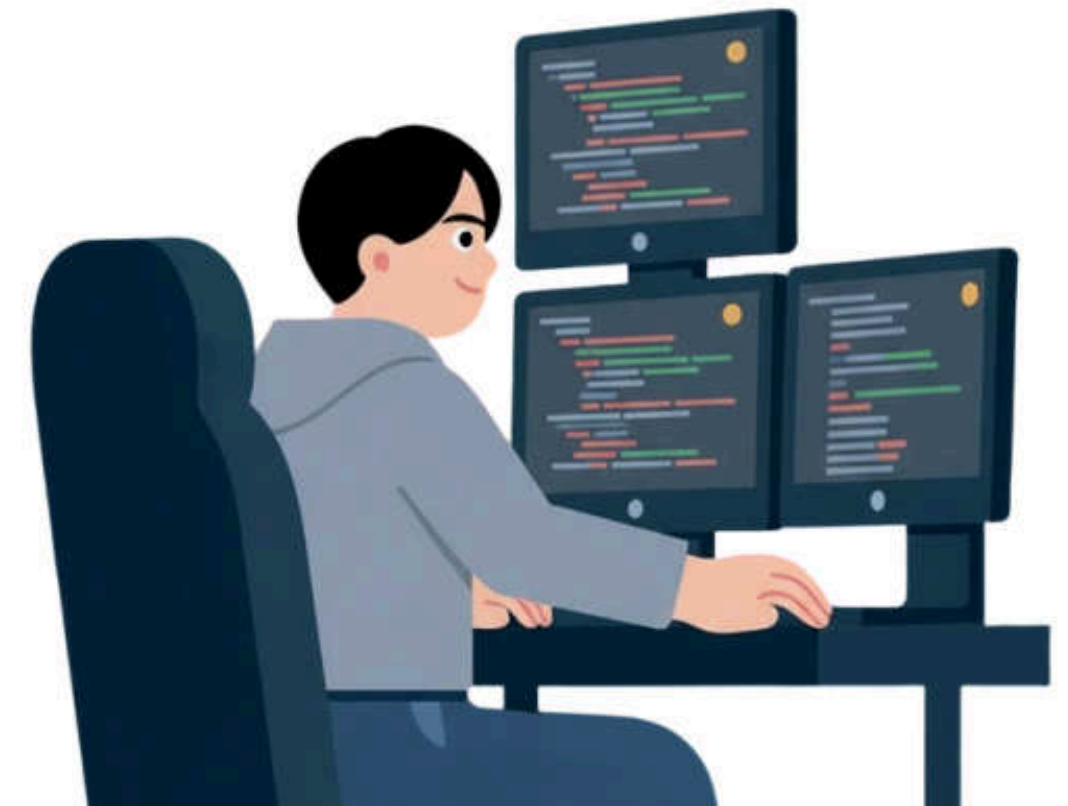
ANALOGIA DA VARIÁVEL

Imagine que várias salas de uma escola utilizem a mesma senha do Wi-Fi.

Seria difícil trocar a senha escrevendo uma nova em vários lugares diferentes.

É mais fácil guardar uma informação principal e fazer todos utilizarem essa mesma referência.

No SASS, a variável funciona de maneira parecida.



3. O QUE É SCSS?

SASS é a ferramenta.

SCSS é uma das sintaxes que podemos utilizar para escrever SASS.

A grande vantagem para quem já conhece CSS é que o SCSS é muito parecido com ele.

Observe:

CSS

```
.produto {  
  background-color: #1a1a1a;  
  padding: 15px;  
}
```

SCSS

```
.produto {  
  background-color: $cor-card;  
  padding: 15px;  
}
```

Perceba que continuamos utilizando:

- `{ }` chaves;
- `:` dois-pontos;
- `;` ponto e vírgula.

Por isso, quem já conhece CSS consegue entender SCSS com facilidade.

5. O QUE SÃO MIXINS?

Imagine que queremos utilizar o mesmo efeito em vários elementos:

```
transition: 0.3s;
```

Nosso site utiliza esse tipo de efeito em vários lugares.

Com SASS podemos criar um **mixin**.

```
@mixin efeito-suave {  
  transition: 0.3s;  
}
```

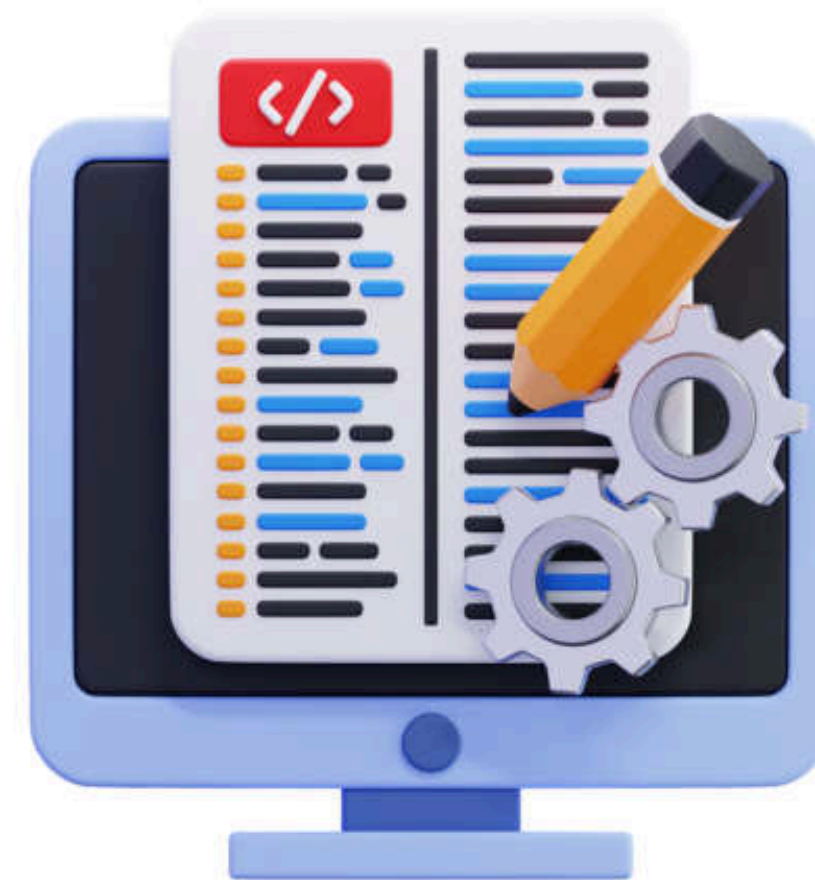
Depois reutilizamos:

```
.produto {  
  @include efeito-suave;  
}
```

```
button {  
  @include efeito-suave;  
}
```

```
.menu a {  
  @include efeito-suave;  
}
```

Assim, evitamos escrever o mesmo conjunto de estilos várias vezes.



O QUE É UM MIXIN?

Um mixin é um bloco de estilos que pode ser reutilizado.

Podemos pensar nele como uma receita.

Criamos uma vez:

```
@mixin efeito-suave {  
    transition: 0.3s;  
}
```

E utilizamos sempre que precisarmos:

```
@include efeito-suave;
```



6. FUNÇÕES E REUTILIZAÇÃO

O SASS também possui funções que podem ajudar a calcular ou gerar valores utilizados nos estilos.

Isso significa que SASS oferece recursos como:

- variáveis;
- mixins;
- funções;
- organização de estilos;
- reutilização de código.

Esses recursos são muito úteis quando o projeto possui muitos arquivos e muitas páginas.

POR QUE ISSO AJUDARIA NOSSO SITE?

Hoje nosso site possui estilos para:

- página inicial;
- produtos;
- contato;
- menu;
- cabeçalho;
- botões;
- cards;
- campos de pesquisa.

À medida que novas páginas forem criadas, mais estilos serão adicionados.

Com SASS podemos criar valores e estilos reutilizáveis.

Por exemplo:

```
$preto: #000;  
$branco: #fff;  
$card: #1a1a1a;  
$tempo-efeito: 0.3s;
```

E utilizar esses valores em diferentes partes do projeto.



7. MANUTENÇÃO DO SITE

Imagine que nossa loja mude sua identidade visual.

Antes:

```
$cor-principal: white;
```

Nova identidade:

```
$cor-principal: #ff6600;
```

Se vários elementos utilizarem `$cor-principal`, podemos alterar a variável principal em vez de procurar cada cor espalhada pelo projeto.

Isso facilita a manutenção.

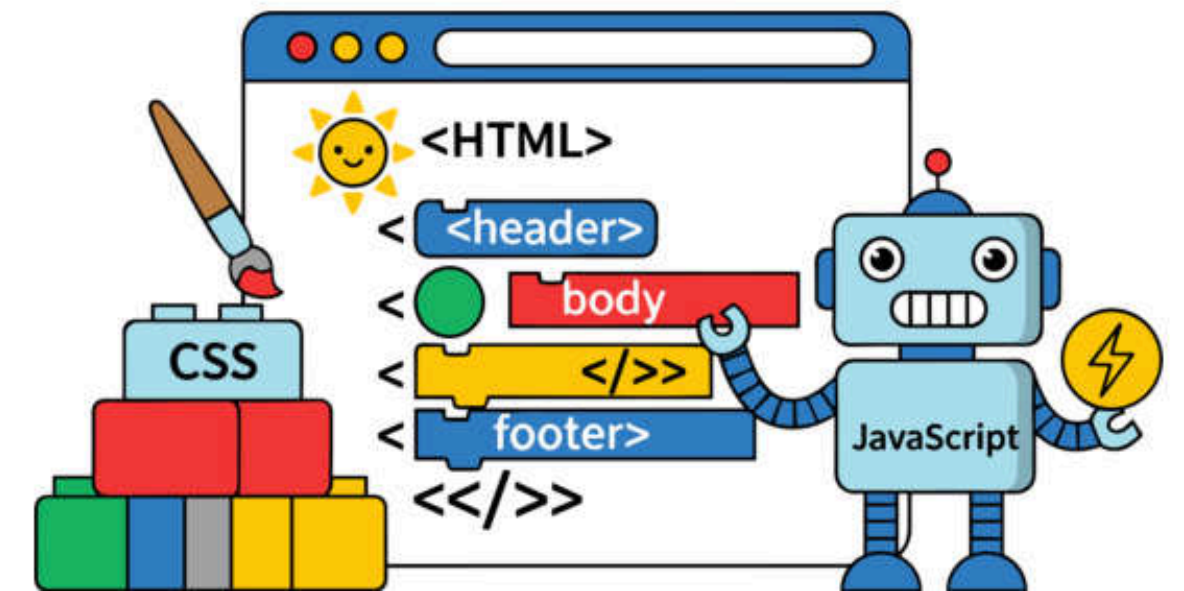
O QUE SIGNIFICA MANUTENÇÃO?

Manutenção é modificar, corrigir ou melhorar um projeto depois que ele já foi criado.

Exemplos:

- trocar uma cor;
- alterar um botão;
- corrigir um estilo;
- adicionar uma nova página;
- modificar o tamanho dos cards;
- atualizar a identidade visual.

Quanto maior o projeto, mais importante fica manter o código organizado.



8. SASS NÃO SUBSTITUI O CSS

Existe uma informação importante:

O navegador entende CSS.

Ele não utiliza diretamente o nosso arquivo SASS/SCSS.

Por isso ocorre um processo chamado:

COMPILAÇÃO

Escrevemos:

`style.scss`



O SASS transforma em:

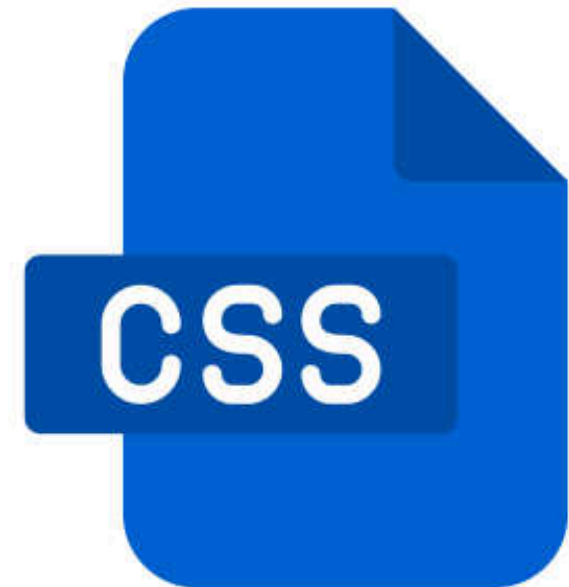
`style.css`



O navegador utiliza o CSS.

Podemos imaginar:

SCSS → compilação → CSS → navegador



9. QUAL PROBLEMA O SASS BUSCA RESOLVER?

Conforme um projeto cresce, o CSS pode possuir:

- muitos valores repetidos;
- estilos espalhados em vários arquivos;
- códigos parecidos;
- dificuldade para alterar vários elementos de uma vez.

O SASS oferece recursos para deixar esse código mais organizado e reutilizável.



9. QUAL PROBLEMA O SASS BUSCA RESOLVER?

Conforme um projeto cresce, o CSS pode possuir:

- muitos valores repetidos;
- estilos espalhados em vários arquivos;
- códigos parecidos;
- dificuldade para alterar vários elementos de uma vez.

O SASS oferece recursos para deixar esse código mais organizado e reutilizável.



APIS EXTERNAS



SEMANA 17
3º BIMESTRE

INTEGRAÇÃO COM APIs EXTERNAS

Uma **API** permite que um sistema utilize informações ou serviços de outro sistema.

Imagine nosso site de vendas querendo mostrar a temperatura de uma cidade.

Nós não precisamos criar um sistema meteorológico.

Podemos pedir essa informação para uma API de clima.

Nosso site faz um pedido → a API responde com os dados.

1. O QUE É UMA API?

API é uma forma de comunicação entre sistemas.

Exemplo:



NOSSO SITE



"Qual a temperatura de São Paulo?"



API DE CLIMA



"22°C"



NOSSO SITE MOSTRA O RESULTADO

Assim, conseguimos adicionar informações externas ao nosso site.

2. OPENWEATHERMAP

O OpenWeatherMap fornece informações sobre o clima.

Por exemplo:

- temperatura;
- condição do tempo;
- cidade;
- umidade;
- previsão do tempo.

Mas para fazer uma requisição normalmente precisamos de uma:

CHAVE DE API

A chave funciona como uma identificação de quem está utilizando o serviço.

Exemplo:

```
abc123xyz456
```

No código ela pode aparecer assim:

```
</> JavaScript
```

```
let apiKey = "MINHA_CHAVE";
```

3. COMO O JAVASCRIPT PEDE INFORMAÇÕES PARA UMA API?

No JavaScript podemos utilizar:

```
⌞ JavaScript
```

```
fetch()
```

O `fetch()` serve para fazer uma requisição e buscar informações externas.

Exemplo simplificado:

```
⌞ JavaScript
```

```
fetch("ENDERECO_DA_API")
```

Podemos pensar:

| `fetch` = buscar.

O QUE SIGNIFICA ASSÍNCRONO?

Quando fazemos uma requisição, a resposta pode demorar alguns instantes.

O JavaScript não precisa travar toda a página enquanto espera.

Ele continua funcionando e recebe a resposta quando ela chegar.

Por isso dizemos que a chamada é **assíncrona**.



4. E SE ALGUMA COISA DER ERRADO?

Imagine que o usuário digite:

Cidade: Asdasdasdas

A API pode não encontrar essa cidade.

Também pode acontecer:

- falta de internet;
- chave de API inválida;
- cidade inexistente;
- serviço indisponível.

Nosso site precisa avisar ao usuário que algo deu errado.

FUNÇÃO `displayError()`

Podemos criar uma função:

`</>` **JavaScript**

```
function displayError(message) {  
    // mostra o erro para o usuário  
}
```

O nome já ajuda:

`display` = mostrar

`error` = erro

Portanto:

`displayError()` significa mostrar uma mensagem de erro.

COMO TUDO FUNCIONA JUNTO?

Imagine uma aplicação de clima:

Usuário digita uma cidade



JavaScript usa `fetch()`



API recebe o pedido



API responde



Site mostra o clima

Mas se ocorrer algum problema:

API apresenta erro



`displayError()`



Mensagem aparece para o usuário

CI/CD E TESTES AUTOMATIZADOS



SEMANA 18
3º BIMESTRE

CI/CD e Testes Automatizados no Front-End

O que vamos estudar?

Quando desenvolvemos um site, não basta apenas escrever HTML, CSS e JavaScript.

Antes de um site ser publicado, é importante verificar se ele está funcionando corretamente.

Imagine o nosso site de vendas:

Alteramos o site

↓

Verificamos se funciona

↓

Corrigimos possíveis erros

↓

Site pode ser publicado

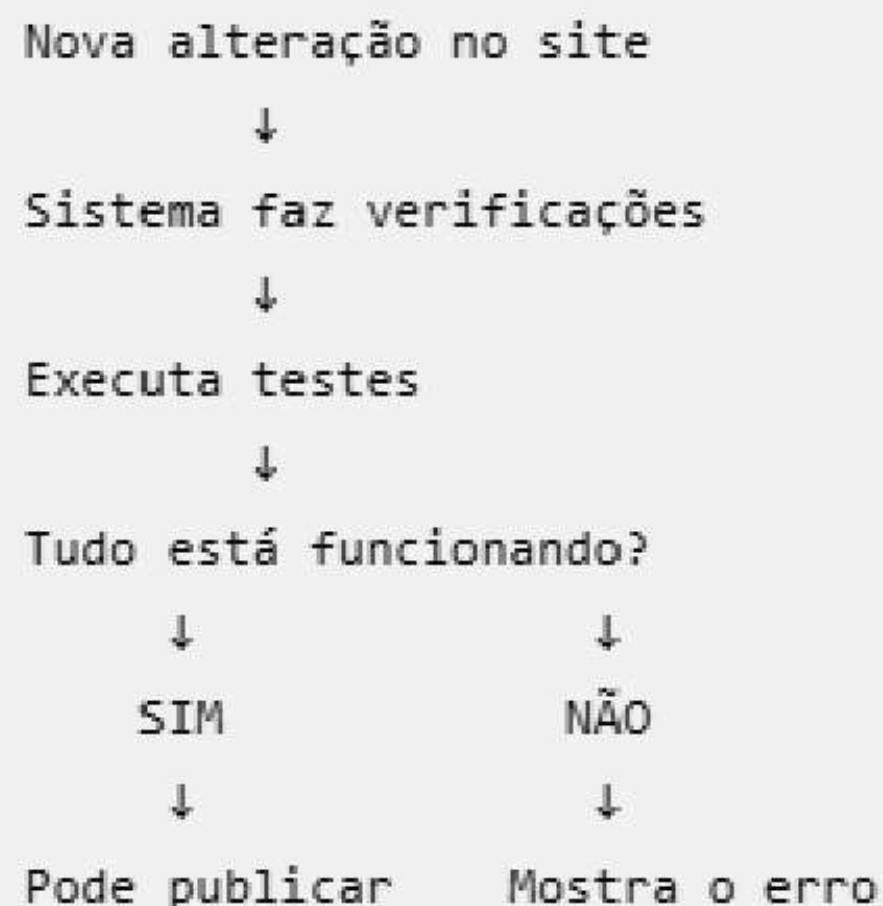
Em empresas, algumas dessas etapas podem ser feitas **automaticamente**.

É aí que entra o **CI/CD**.

O que é CI/CD?

CI/CD é uma forma de **automatizar etapas de verificação e publicação de um projeto.**

Imagine que alguém alterou a página de produtos do site:



Assim, não é necessário depender somente de uma pessoa verificando tudo manualmente.

O principal objetivo do CI/CD é:

automatizar testes e deploy, melhorando a qualidade e a agilidade do desenvolvimento.

O que é Pipeline?

Pipeline é o nome dado à sequência de etapas automáticas pelas quais o projeto passa.

Pense como uma esteira:

```
SITE
↓
TESTES
↓
VERIFICAÇÃO
↓
PUBLICAÇÃO
```

Por exemplo, no site de vendas:

```
Alteramos um botão
↓
Testes verificam o site
↓
Não encontrou problema
↓
Site pode ser publicado
```

Então:

Pipeline = sequência de etapas automáticas.

O que são Testes Automatizados?

Quando vocês alteram o site, normalmente fazem testes manualmente.

Por exemplo:

- Abrir a página
- Clicar nos botões
- Testar os links
- Ver se as imagens aparecem
- Testar a pesquisa

Nos **testes automatizados**, quem realiza determinadas verificações é o próprio sistema.

Por exemplo:

```
graph TD; A[O código foi alterado] --> B[Teste automático]; B --> C[Está funcionando?];
```

Se encontrar um problema, ele pode impedir que o processo continue.

Por isso, dentro de um pipeline de CI/CD, a etapa mais indicada para verificar a qualidade do código é:

Testes automatizados.

O que é Deploy?

Deploy significa colocar o site em funcionamento para os usuários.

De forma simples:

Site sendo desenvolvido

↓

Site pronto

↓

DEPLOY

↓

Site publicado

Então não confundam:

TESTES

→ verificam se está funcionando

DEPLOY

→ coloca o site em funcionamento/publica

E o que é Build?

Build é a preparação do projeto para que ele possa funcionar ou ser publicado.

Para esta aula, basta lembrar:

BUILD

→ prepara

TESTES

→ verificam

DEPLOY

→ publica

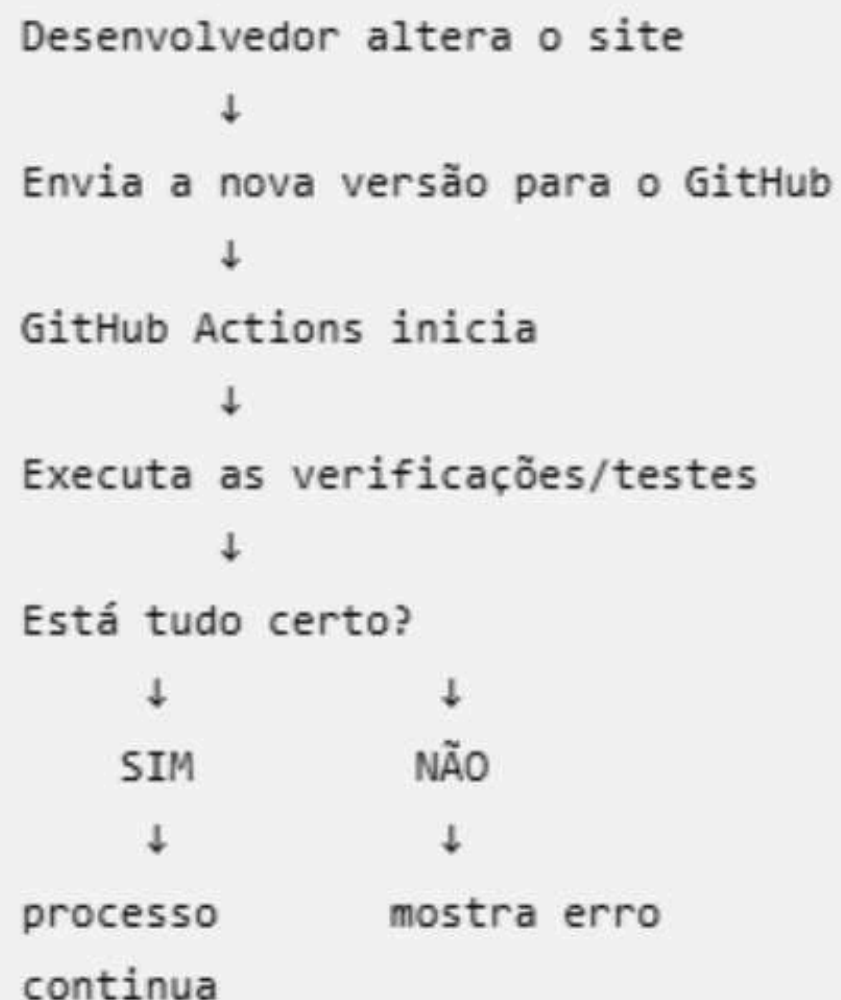
Como o GitHub entra nesse processo?

Para que os testes aconteçam automaticamente, normalmente utilizamos uma plataforma que recebe o código e executa essas verificações.

Um exemplo é o **GitHub**.

No GitHub existe um recurso chamado **GitHub Actions**, que pode ser configurado para executar tarefas automáticas quando o código do projeto é atualizado.

Exemplo:



Ou seja:

O CI/CD é o processo de automação.

O GitHub pode ser uma das ferramentas usadas para executar esse processo.

Mas como o código chega até o GitHub?

Antes de enviar uma alteração, existem algumas etapas com o **Git**.

Não precisamos aprender Git inteiro agora. Basta entender este fluxo:

Alteramos os arquivos do site

↓

`git add .`

↓

`git commit`

↓

`git push`

↓

Código enviado ao GitHub

Mas como o código chega até o GitHub?

Antes de enviar uma alteração, existem algumas etapas com o **Git**.

Não precisamos aprender Git inteiro agora. Basta entender este fluxo:

Alteramos os arquivos do site

↓

`git add .`

↓

`git commit`

↓

`git push`

↓

Código enviado ao GitHub

O que faz o `git add`?

O comando:

`</> Bash`

`git add .`

serve para preparar os arquivos alterados que queremos incluir na próxima atualização registrada pelo **Git**.

Exemplo:

Você alterou:

```
produtos.html  
produtos.css  
produtos.js
```

Ao executar:

```
</> Bash  
  
git add .
```

o Git entende:

```
"Esses arquivos estão prontos  
para entrar no próximo registro."
```

Ligando tudo

Imagine que alteramos o botão **Ver Produto** no nosso site:

Alteramos o código

↓

`git add .`

Preparamos os arquivos

↓

`git commit`

Registramos a alteração

↓

`git push`

Enviamos para o GitHub

↓

GitHub Actions

↓

Testes automatizados

↓

Tudo funcionando

↓

Deploy / publicação

Então, para esta aula, basta lembrar:

Git

→ controla alterações no código

`git add .`

→ prepara todos os arquivos alterados

GitHub

→ pode receber o projeto

GitHub Actions

→ pode executar verificações automáticas

Testes automatizados

→ verificam se o código está funcionando

Deploy

→ publica o projeto

- **MOBILE-FIRST**
- **ACESSIBILIDADE E ANIMAÇÕES INTERATIVAS**
- **SERVIÇOS DE AUTENTICAÇÃO E MONITORAMENTO**



**SEMANAS
19, 20 e 21
3º BIMESTRE**

CSS Grid

O **CSS Grid** é utilizado para organizar elementos da página em linhas e colunas.

Para transformar uma área em Grid, usamos:

```
</> CSS
```

```
display: grid;
```

Depois podemos definir como as colunas serão distribuídas.

Exemplo:

```
</> CSS
```

```
grid-template-columns: 1fr 2fr 1fr;
```

Nesse caso teremos:

1ª coluna	2ª coluna	3ª coluna
1fr	2fr	1fr

Mobile-First

Mobile-First significa criar primeiro o layout pensando em telas pequenas e depois adaptar para telas maiores.

Exemplo:

`</>` CSS

```
.layout {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
.coluna-lateral {  
  display: none;  
}
```

No celular temos apenas uma coluna.

Quando a tela chegar a 960 px, podemos alterar o layout:

`</>` CSS

```
@media (min-width: 960px) {  
  
  .layout {  
    grid-template-columns: 1fr 2fr 1fr;  
  }  
  
  .coluna-lateral {  
    display: block;  
  }  
  
}
```

Assim, a partir de 960 px, aparecem as três colunas e a coluna lateral fica visível.

Para lembrar: `display: grid` ativa o Grid; `1fr 2fr 1fr` cria três colunas com o centro maior; `960px` o ponto utilizado nesta atividade para mostrar a coluna lateral.

SEMANA 2 — Acessibilidade e Animações Interativas

O que é acessibilidade?

Um site acessível deve conseguir ser utilizado também por pessoas que dependem de tecnologias assistivas, como leitores de tela.

Os atributos ARIA fornecem informações extras para essas tecnologias.

Exemplo:

 HTML

```
<button aria-label="Abrir menu">  
  ≡  
</button>
```

O `aria-label` ajuda o leitor de tela a entender o que aquele botão faz.

ARIA não serve para deixar o site mais bonito. Seu objetivo é ajudar tecnologias assistivas a compreenderem a interface.

aria-live e WAVE

O `aria-live` pode ser utilizado quando alguma informação da página muda sem o usuário trocar de tela.

Exemplo:

 HTML

```
<p aria-live="polite">
  Produto adicionado com sucesso.
</p>
```

O leitor de tela poderá anunciar essa alteração ao usuário.

Para verificar problemas de acessibilidade também podemos utilizar o **WAVE**.

Ele ajuda a encontrar problemas como:

```
Texto com pouco contraste
  ↓
Difícil de ler
  ↓
Problema de acessibilidade
```

Para lembrar: ARIA fornece informações para tecnologias assistivas; `aria-live` anuncia conteúdos que mudam; WAVE pode ajudar a identificar contraste insuficiente.

SEMANA 3 — Serviços de Autenticação e Monitoramento

Login com Google

Em vez de o site criar e armazenar uma nova senha para cada usuário, podemos permitir:

ENTRAR COM GOOGLE

Nesse processo, o usuário utiliza sua própria conta Google para realizar a autenticação.

Fluxo simplificado:

Usuário clica em
ENTRAR COM GOOGLE

↓

Google realiza o login

↓

Google confirma a identidade

↓

A aplicação recebe as
informações necessárias

Isso permite utilizar uma forma de autenticação já fornecida pelo Google.

Google Cloud e JWT

Para configurar o login, é necessário utilizar o Google Cloud Console.

Entre as configurações importantes estão:

```
client_id
+
URI de redirecionamento autorizado
```

O `client_id` identifica a aplicação para o Google.

Depois do login, a aplicação pode receber um token *JWT* contendo informações do usuário.

Em aplicações React, uma biblioteca como:

```
</> JavaScript
jwt-decode
```

pode ser utilizada para decodificar o *JWT* e obter as informações presentes nele.

Exemplo conceitual:

TOKEN JWT

↓

jwt-decode

↓

Nome

E-mail

Outras informações

Ela não cria nem criptografa o token. Sua função, nessa situação, é decodificá-lo.